

Experiment 4 — Google Stock Price Prediction using RNN/LSTM

Aim

The aim of this project is to predict future Google stock prices using:

Recurrent Neural Network (RNN) with LSTM

using TensorFlow and Keras.

Problem Type

This is:

Time Series Prediction

because stock prices depend on previous values over time.

Overall Flow

Import Libraries

↓

Download Google Stock Data

↓

Preprocess and Scale Data

↓

Create Sequences

↓

Build LSTM Model

↓

Train Model
↓
Predict Stock Prices
↓
Calculate RMSE
↓
Predict Future Prices
↓
Display Graphs

Dataset

Google stock data downloaded using:

yfinance

Dataset contains:

- Open price
- High price
- Low price
- Close price
- Volume

The model mainly uses:

Closing Price

Why Stock Prediction Uses RNN/LSTM?

Stock prices are sequential data.

Today's price depends on:

- yesterday's price
- previous trends

LSTM remembers previous information.

Data Scaling

Data is scaled using:

MinMaxScaler

Formula:

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Why Scaling?

Advantages:

- faster convergence
- stable training
- prevents huge value domination

Sequence Creation

The model uses:

60 previous days

to predict next day stock price.

Example:

Day 1 → Day 60

↓

Predict Day 61

This is called:

Sliding Window Technique

LSTM Architecture

Model contains:

- Multiple LSTM layers
 - Dropout layers
 - Dense output layer
-

What is LSTM?

Full form:

Long Short-Term Memory

LSTM is advanced RNN architecture.

It remembers long-term dependencies in sequential data.

Why LSTM is Better than Normal RNN?

Normal RNN suffers from:

Vanishing Gradient Problem

LSTM solves this using:

- memory cells
 - gates
-

LSTM Layer

Example:

```
LSTM(50, return_sequences=True)
```

50 Units

Means:

50 memory neurons

are used.

return_sequences=True

Means:

- full sequence output passed to next LSTM layer

Used when stacking multiple LSTM layers.

Dropout Layer

Example:

Dropout(0.2)

Purpose:

- prevents overfitting

Randomly disables neurons during training.

Dense Output Layer

Final Dense layer predicts:

Next Stock Price

Since this is regression:

- only one neuron used

Usually no activation or linear activation used.

Loss Function

Model uses:

Mean Squared Error (MSE)

Formula:

$$MSE = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

Purpose of MSE

Measures prediction error.

Lower MSE = better predictions.

Optimizer

Adam Optimizer

Purpose:

- updates weights efficiently
 - minimizes loss
 - speeds up learning
-

Epoch

One epoch means:

complete training dataset passed once through model.

Example:

epochs = 50

means:

model studies dataset 50 times.

Batch Size

batch_size = 32

Training data divided into small batches.

Advantages:

- efficient memory usage
 - smoother learning
 - faster computation
-

Backpropagation Through Time (BPTT)

Special backpropagation used in RNN/LSTM.

Error moves backward through:

- neurons
- time steps

This improves sequence learning.

RMSE

Model performance evaluated using:

Root Mean Square Error

Formula:

$$RMSE = \sqrt{\frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2}$$
$$MSE = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

Meaning

Lower RMSE means:

- predictions are closer to actual stock prices.
-

Future Prediction

The trained model predicts future stock prices using:

- latest 60-day sequence
 - recursive prediction method
-

Graphs

Graphs show:

- actual stock prices
- predicted stock prices
- training loss

If prediction curve closely follows actual curve:

- model is performing well.
-

Final Conclusion

This project uses LSTM-based Recurrent Neural Networks for Google stock price prediction. Historical stock prices are scaled and converted into sequential data using sliding windows. Multiple LSTM layers learn temporal dependencies, while Dropout prevents overfitting. The model is trained using Adam optimizer and MSE loss function to predict future stock prices accurately.